

CS 1050 Technical Documentation

Click table of contents below to refresh after adding more headings

[Overview](#)

[General Resources](#)

[SD01 Software Development and Tools](#)

[Version Control](#)

[IDE and Debugging](#)

[Analyze requirements](#)

[Design and Testing Plan](#)

[Implement Quality Documented Code](#)

[Module 1: Programming Fundamentals and Tools](#)

[D.I.A. variables](#)

[Constance/variables](#)

[Order of operations, integer division and the modulus operator.](#)

[Select appropriate numeric primitive data types \(int, double\) based on the type of data and required precision](#)

[Mixed number \(doubles and integers\) operations](#)

[Apply assignment operators and basic arithmetic operators to form valid expressions.](#)

[Implicit and explicit conversion rules and casting](#)

[Memory allocation based on data type and potential overflow errors during program execution](#)

[Standard output using System class and print methods](#)

[Identify different types of Errors](#)

[Module 2: Predefined Classes, Methods, and Decision Structures](#)

[Heading 2 Example](#)

[Heading 3 Example](#)

[Module 3: Loops and Methods](#)

[Heading 2 Example](#)

[Heading 3 Example](#)

[Module 4: Arrays](#)

[Heading 2 Example](#)

[Heading 3 Example](#)

[Module 5: Classes and Objects](#)

[Heading 2 Example](#)

[Heading 3 Example](#)

Overview

When you add information in your own words demonstrating your understanding. Do not use exact words from lectures or images from lectures.

DOs and DON'Ts	
DO: Provide clear and simple explanations in your own words (be concise)	• Use short sentences, small paragraphs • Use lists and tables to organize information • Use standard industry terms but explain in simple language • Use AI Tools to help you learn but use lectures first
DON'T copy my explanations from the slides as yours.	
DO Use Code Snippets and Visuals to reduce lengthy explanations:	• Include examples you explored or created • Include comments to describe parts of code • Include Screenshots such as from your debugger • Include your own drawings such as for memory, arrays, etc. • Include images you have AI create (cite that AI created) • Include images from the Internet with the link
DON'T use my slides as your image and explanations	
DO Include Resources that give steps or help explain concepts	• You can include resources given in the lecture or find your own but ◦ Include website link with summary ◦ Include links to videos with summary of content
DON'T use AI to create your technical documentation.	

General Resources

- Your first source should always be your course lecture slides and documents or links to information in the lectures
 - [F26 CS1050 Student Course Resources](#)
 - [Java Environment Setup](#)
 - [Introduction to Programming with Java](#) - use the sections shared with you to practice ideas from the lecture.
- Next resource is to see Deb or learning assistance during office hours
- If you're unsure whether a source is trustworthy, ask your instructor first.
- ⚠ Avoid These Sources
 - TPointTech <https://www.tpointtech.com/>
 - W3Schools – Java, syntax, data types, etc.: <https://www.w3schools.com/java/>
 - Sites without a known author or date
 - AI-generated blogs that do not cite documentation
 - Use of AI should be last as it will probably give you information you haven't learned or understand. It is expected you will be

documenting concepts from class and book using your code examples.

Other Tools you should use:

- Eclipse
- [Visualize Java Code](#) - used to step through code and edit code

SD01 Software Development and Tools

Version Control

- Use version control management tools (Git/GitHub) to manage and document development with GitHub Desktop.

IDE and Debugging

- Write and debug programs in IDE (Eclipse)

Analyze requirements

Analyze requirements(user stories and acceptance criteria) to break problems into smaller testable tasks

Design and Testing Plan

- Apply Test-Driven Development (TDD): Create unit test cases, design algorithms in pseudocode, implement code (structured, readable and documented), debug and unit test incrementally
- Develop a documented plan by creating unit test cases and algorithm designs
- Use UML to design and implement multiple class programs.

Implement Quality Documented Code

- Apply Java naming conventions for variables, constants, methods, and classes.
- Write comments at top of file and inline comments to document code.
- Use constants and variables and avoid hard coding literals
- Document method code following javadoc commenting
- Use Single Responsibility Principle (SRP) and Do Not Repeat Yourself (DRY) to implement modular, readable and testable solutions

Module 1: Programming Fundamentals and Tools

Add the information to demonstrate the following objectives and organize with headings:

Add the information to demonstrate the following objectives and organize with headings:

- Declaring, initializing, and assigning variables.
- Compare constants and variables
- Order of operations, integer division and the modulus operator.
- Memory allocation based on data type and potential overflow errors during program execution
- Select appropriate numeric primitive data types (int, double) based on the type of data and required precision.
- Apply assignment operators and basic arithmetic operators to form valid expressions.
- Implicit and explicit conversion rules and casting
- Mixed number (doubles and integers) operations
- Standard output using System class and print methods.
- Identify different types of Errors
 - Syntax error. Include a screenshot of your IDE showing there is a syntax error
 - Runtime error. Include a screenshot of your IDE showing
 - Logical errors.

D.I.A. variables

Declaring a variable is simply saying that variable exists it's like calling attention to an object that is been created without stating anything else about it like pointing to a box and saying it exists encoding format it looks like this:

```
Double numberCount;
```

The program now knows that variable exists and what type of variable it is but does not have a value for it and will hold that variable for the rest of the code.

Initializing a variable is the second step of declaring a variable. What this looks like is assigning a value To a declared variable this does not need to be done separately from declaring the variable but it cannot be done without First declaring either

The format of doing declaration and initialization individually looks like the following

```
//The step below is the initialization of the variable number count  
Double numberCount;
```

```
//The step below is the Declaration of the variable number count  
numberCount = 20.77;
```

In the above demonstration we can see why initialization is mandatory in this two-step process because without it we are assigning a value to a variable that does not exist which in code as well as real life just makes you look crazy

However this two-part process can be done in the same line of code and it would look something like this

```
Double numberCount =20.77;
```

Assigning a variable would look similar to the processes Above however assigning works is modifier and is not exactly the same as declaration for example if a variable already has an existing value assigned to it through declaration it can then be changed again via similar methods below:

```
//This method of assigning a variable modifies the existing value by using math  
resulting in the sum of the operation becoming the new value for the variable  
numberCount = numberCount + 2406;
```

Or

```
//This method modifies the variable by directly getting rid of its existing value  
and replacing it with a new specified value without using any numerical  
operation to modify other than specifying which number we would like to  
change the value of the variable too  
numberCount = 4201;
```

I'm sure there are other methods of assigning value to a variable but these are the first two I could think of off the top of my head but as you can see assigning a value to a variable can be done later on in the code as a means of reusing a variable for a different reason or modifying it to streamline the code in a way that makes sense for whatever operation is being done.

Constance/variables

A constant is like a variable in many ways it holds the value that we want the program to recognize however once declared it cannot be modified throughout the code hence the term constant the use case for this is as follows referencing back to a constant multiple times in the code without worrying that it'll be modified later on ensures that that value can be used as the basis for multiple operations that require it.

example

```
//It is important to note the documentation of a constant is different than a variable in that it uses all caps as well as underscores between words used for the constant and an additional Declaration of final is used to classify the variable as a constant, ensuring that it can't be modified later on by accident  
Final Int VAR_ONE= 4;
```

a variable is essentially an empty box that can be filled with whatever value we place in it of course assuming that we initialize the variable for type properly for example using words versus numbers or specifying cases for decimal use a variable can be changed throughout the program and modified as the code progresses if set up properly this fluidity allows us to reuse variables throughout our code and streamline our thought process.

Order of operations, integer division and the modulus operator.

the way math works in code is the same as the normal order of operations PEMDAS and as such will calculate In that order, However for complex operations that require more than the set list that order of operations access more elaborate coding is necessary

Math needing to be broken down	Step by step of how the computer reads it
$4(45 + 3) - 8$	First: $45 + 3 = 48$ Second $48 * 4 = 192$ Three $192 - 8 = 184$

The above demonstrates that the program isn't simply reading from left to right but using PEMDAS to categorically run the math however for more complex operations that require interpolation the more complex process will be needed

Integer division is an important topic because if we have two variables Defined as integers and we try to divide them we will get an answer however any remainder or decimal point will be cut off as integers specifically identified as a whole number this can be remedied by one of the variables in the division operation being defined as a double allowing the resulting answer to be a double as well and does not cut off the decimal point.

Code written	How end var will be stored
<pre>//One of the two integers that will be used in the mathematics operation later on Int numberOne = 12; //The second integer that will be used in the mathematical operation later on Int numberTwo = 6; //an empty declared and initialized integer that will hold the results of the previous two integers after the operation is performed Int numberThree; numberThree = numberTwo / numberOne;</pre>	<pre>//Though not technically a wrong answer the full correct answer is 0.5 however due numberThree not being a double the full value was not stored in memory Console: 0 //Theoretically, if the variable number three was stored properly as a double the computer would store the results of the operation as below Console: 0.5</pre>

However the modulus operator Works in the exact inverse of what happens with integer division and if used will only result in the remainder or decimal value of the division operation so for all intensive purposes the modulus operator works the same as standard division but regardless of if the inputted variables are integer or double will result in the decimal value of the operation

Code written	How end var will be stored
<pre>//One of the two integers that will be used in the mathematics operation later on Int numberOne = 12; //The second integer that will be used in the mathematical operation later on Int numberTwo = 6; //an empty declared and initialized integer that will hold the results of the previous two integers after the operation is performed Int numberThree; numberThree = numberTwo % numberOne;</pre>	<pre>//Though not technically a wrong answer the full correct answer is 0.5 however due Using the modulus operator to run the math everything but the remainder was not stored in memory meaning that the resulting decimal is the only thing displayed Console: 5 //Theoretically, if the variable number three was stored properly as a double and the Division symbol was used instead the computer would store the results of the operation as below Console: 0.5</pre>

Select appropriate numeric primitive data types (int, double) based on the type of data and required precision

Integer: An integer is a type of variable that defines a whole number and as such cannot exist in a fraction or as demonstrated as a decimal. the example below shows a few ways in integer can look in a program

```
Int example = 20;
```

Or

```
Int example;
```


Example = 20;

Either example above works for initializing, declaring and assigning an integer, which for this example would be titled “example” This format would work for other kinds of variables as well such as double As will be demonstrated below

double:

Double example = 20.77;

Or

Double example;

Example = 20.77;

Mixed number (doubles and integers) operations

Mixed number operations operate on a basic principle assuming operation involves two double variables if we are dividing these two variables the resulting math will come out as a double And the same situation occurs if we do the same operation with two integer variables however if we have one integer variable and one double variable the resulting math will be a double variable given that we have now established by using a double that the extra parameter that differentiates a double from an integer now applies to the sum of the operation this is always the case for integers and doubles as long as there is at least one double

Example for int/int:

```
Int number2 = 2;
```

```
Int number3 = 3;
```

```
Int total;
```

```
Total = number3 / number2;
```

```
system.out.println(total);
```

Console:

0

Example for int/double:

```
Double number2 =2;
```

```
Int number3 = 3;
```

```
Int total;
```

```
Total = number3 / number2;
```

```
system.out.println(total);
```

Console:

1.5

Despite being initialized as an integer the resulting code comes out as a double

Apply assignment operators and basic arithmetic operators to form valid expressions.

+	Use for accumulating the total of two values
-	Used for subtracting two or more values
/	Use when you want to divide two values however be careful when assigning your variables integer or double values given that which variables are in their operation will affect the resulting total
%	Use only when you want to get the remainder or decimal value of a division operation
*	Used for multiplication if you want to multiply two or more variables together use this operator
()	Remember program will solve in the order of PEMDAS parentheses are used to isolate numerical values in your operation to ensure things are done in the correct order

Implicit and explicit conversion rules and casting

Implicit Type casting(implicit) like a double being DIA on the surface as 3 but in memory is stored as 3.0. And essentially operates as the computer employing certain rules that are generally always followed for the specific types of casting being used Such as establishing A double as the value 3.0 Instead of just three despite them conveying the same thing It's creating the space for that memory so that the computer knows what it's looking at when it references back to it later on.

Exp:

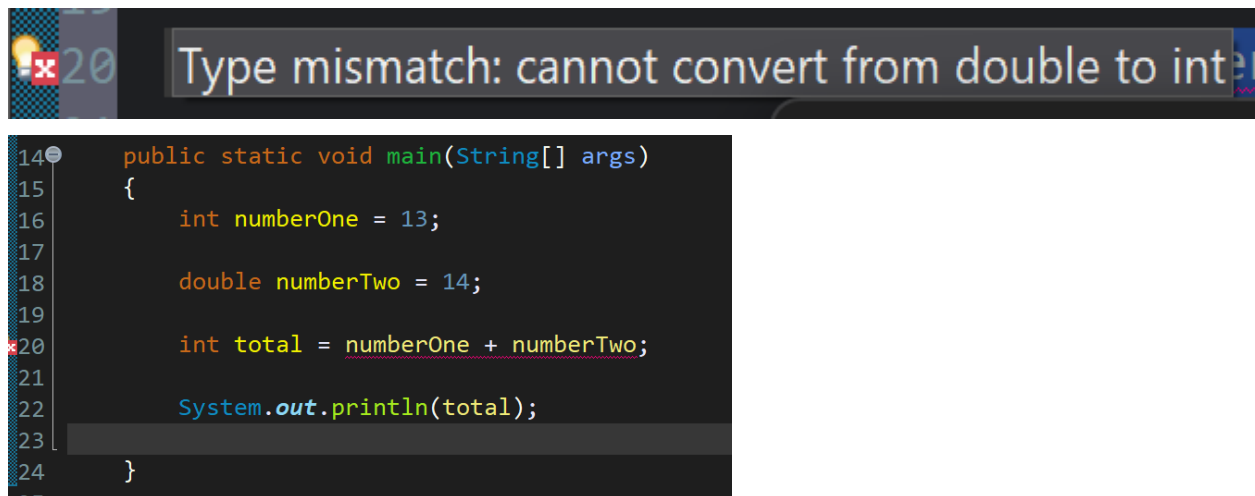
Double number = 3;

Memory

Explicit Type casting(explicit) like after when making a variable you *have* to say what it is, double/int/string Explicit type casting is essentially what we need to tell the computer so that it can make implications based off the foundation of the rules we apply when we are writing code such as Declaring a variable so that if it is referenced later on in the code the computer knows that it exists in memory and or identifying variable types such as integer double or string So that the computer knows what to classify the declared variable as

Memory allocation based on data type and potential overflow errors during program execution

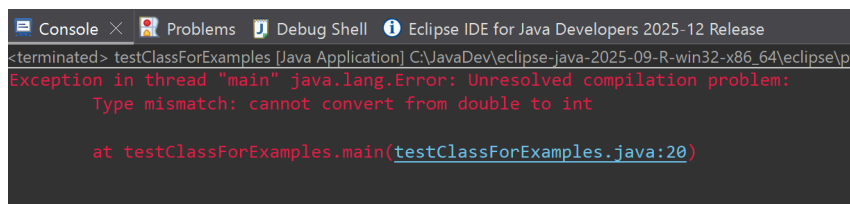
Memory allocation works differently for different variable types, such as integers or doubles for example, the computer will store the value of an integer differently from a double such as in the example below



```

14 public static void main(String[] args)
15 {
16     int numberOne = 13;
17
18     double numberTwo = 14;
19
20     int total = numberOne + numberTwo;
21
22     System.out.println(total);
23
24 }
  
```

//Essentially, what is happening is the memory allocated to the integer cannot store the extra memory that is allocated to the double, thus causing the Overflow error



```

<terminated> testClassForExamples [Java Application] C:\JavaDev\eclipse-java-2025-09-R-win32-x86_64\eclipse\p
Exception in thread "main" java.lang.Error: Unresolved compilation problem:
    Type mismatch: cannot convert from double to int
    at testClassForExamples.main(testClassForExamples.java:20)
  
```

For example, if an integer was assigned the value of 5, the computer would then store said value exactly as is however, if a double was assigned to the value of five, the

computer would read that value as 5.0 as seen below

Code written	How its read
Int numberOne = 4; System.out.println(numberOne);	4

Code written	How its read
Double numberOne = 4; System.out.println(numberOne);	4.0

Standard output using System class and print methods

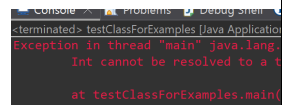
Standard output is whenever the user designates that the computer output some variable or other form of information stored within the system this is most frequently done by calling to the system class and requesting an output followed by a parameter of what should be printed said result is then printed to the console. Typical means of this are listed below

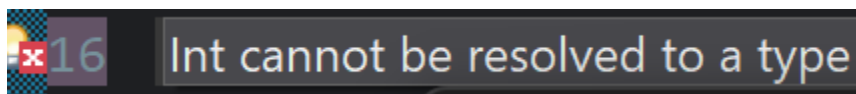
means	What it does
System.out.println();	This output method displays whatever Variable or string is referenced within the parentheses and then starts a new line
System.out.printf();	This output method formats whatever Variable is parade did in parentheses if followed by the proper Conversation rule
System.out.print();	In this output method displays whatever variable or string is within the parentheses

The difference between each of these system print methods is settled but have their own individual functionality and use case specified always by what follows the word print in the command statement system.out.print[blank]

Identify different types of Errors

Syntax errors: Send text errors or grammatical errors in the form of the programming language for example if we're declaring a variable and misspell the variable type we will get a syntax error, such as misspelling int for some other variation of the word as seen below

correct	incorrect	Incorrect syntax result
<code>int numberOne = 12;</code>	<code>Int numberOne = 12;</code>	



Runtime errors: Runtime is occur when the computer receives an input that it was not expecting For example if a program asks the user for a int and instead inputs a string The computer will then throw a runtime error as exhibited below

```
Enter your first grade: no
Exception in thread "main" java.util.InputMismatchException
    at java.base/java.util.Scanner.throwFor(Scanner.java:977)
    at java.base/java.util.Scanner.next(Scanner.java:1632)
    at java.base/java.util.Scanner.nextInt(Scanner.java:2297)
    at java.base/java.util.Scanner.nextInt(Scanner.java:2251)
    at GEM01Calculation.main(GEM01Calculation.java:41)
```

Runtime error	No runtime error
Console: Enter any whole number:	Console: Enter any whole number:

User input:no	User input:12
Console: ERROR	Console: Your number is 12!

Logical errors: Logical errors occur when no syntax or runtime errors are present and the program runs as executed, but data is still lost in the process, for example, when integer division is used but the results needed is a double the resulting math will truncate the answer to the nearest whole number and exclude the decimal, leading to a logic error Example below:

Code written	How end var will be stored
<pre>//One of the two integers that will be used in the mathematics operation later on Int numberOne = 12; //The second integer that will be used in the mathematical operation later on Int numberTwo = 6; //an empty declared and initialized integer that will hold the results of the previous two integers after the operation is performed Int numberThree; numberThree = numberTwo / numberOne;</pre>	<pre>//Though not technically a wrong answer the full correct answer is 0.5 however due numberThree not being a double the full value was not stored in memory And the full correct answer was truncated off of what the console produced / stored in memory Console: 0 //Theoretically, if the variable number three was stored properly as a double the computer would store the results of the operation as below Console: 0.5</pre>

Module 2: Predefined Classes, Methods, and Decision Structures

Add the information to demonstrate the following objectives and organize with headings:

- Using built-in classes (System, Scanner, String, Character, Math) and their methods; parameters and return values
- Distinguish between characters (char) and Strings.
- String as an array of characters and use String class methods.
- Use Boolean values and relational operators to compare data.

- Distinguish between if-else, multi-way if, and nested if statements to control program flow.
- Scope of variables
- Use logical operators (&&, ||, !) to combine conditions.
- Use of Switch statements for multi-way selection

Using built-in classes (System, Scanner, String, Character, Math) and their methods; parameters and return values Distinguish between characters (char) and Strings.

System is a call to the Java library of preset classes, for example system.outprint tell the program to call the system print out whatever is in the parentheses of the method

```
System.out.println("understood your ticket will cost $" + rowOne);
```

Scanner is it cold to an existing Java class that basically accepts user input from the keyboard and can be stored as a variable an example would be when it is

```
import java.util.Scanner;
```

brought in to the program as

declared as `Scanner input = new Scanner(System.in);` We are signing input to recognize the scanner class so we can use it later when we are accepting input from the keyboard with the command

```
userSection = input.next().charAt(0);
```

As for string char and math those are individualistic classes used for inputting actual text and simplifying mathematical equations so that programs requiring them can be condensed string and Char both store similar types of data the only difference between them is that a string is a daisy chain of chars in order and are stored in memory as ints Which is what it allows commands such as charAt to work properly essentially it's looking at a chain of numerical values in plucking the one at the coordinates specified within the parentheses

Use Boolean values and relational operators to compare data.

I'm not entirely sure what this category is asking but I can't explain Boolean values and how relational operators work full in values essentially operate on a true false basis meaning they're either a zero or one relation to operators are

modifiers of this core logic meaning or(||), and(&&), not(!=) and so on Which allow the true false logic table to change in relation to what's applicable for any given line of code for example the or modifier allows this example of code to work in one line as opposed to multiple if statements

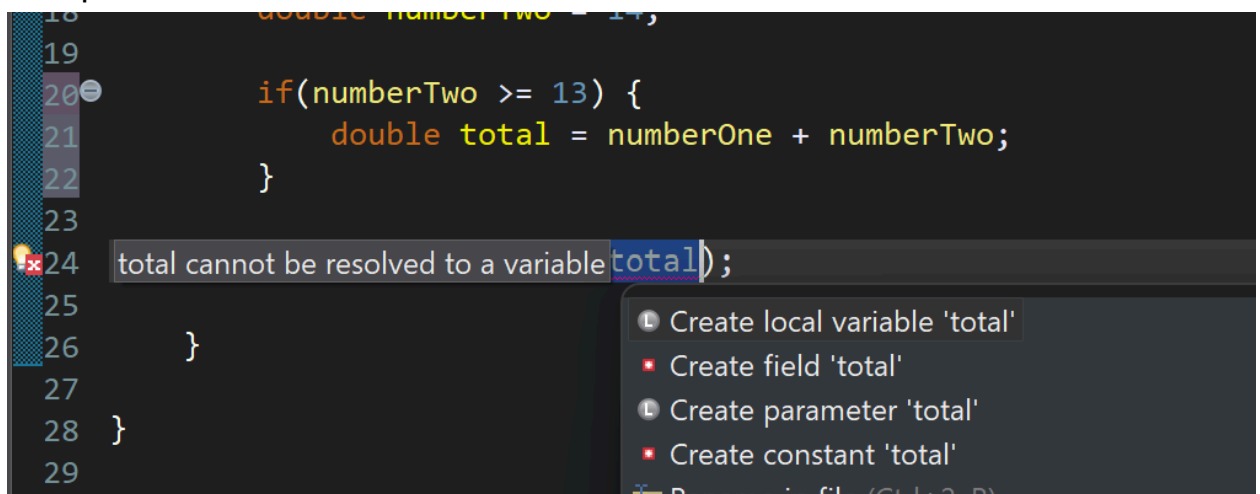
```
if ((userSection == sectionF) || (userSection == sectionC) || (userSection == sectionS  
    || (userSection == sectionU))
```

Instead of having to do multiple if statements I can use the or operator to compare different statements and create a unified of statement where if any of the above statements are true it runs the command

Scope of variables

The scope of variables Extends to the data type they're declared in for example if you declare an end you cannot store a double in that variable it will simply be truncated when the program is run.

The scope for if statements basically extends to everything within the brackets, meaning that everything within that if statement only exists there. If you declare a variable in an if statement outside of that if statement, it does not exist, so where possible, try to initialize variables before you use enough statement examples below



```
18 double numberTwo = 14;  
19  
20 if(numberTwo >= 13) {  
21     double total = numberOne + numberTwo;  
22 }  
23  
24 total cannot be resolved to a variable total);  
25  
26 }  
27  
28 }  
29
```

- Create local variable 'total'
- Create field 'total'
- Create parameter 'total'
- Create constant 'total'
- Rename in file (Ctrl+2, D)

Because the total is declared OUTside the scope the program doesn't know it exists

Distinguish between if-else, multi-way if, and nested if statements to control program flow.

There are different types of these statements: if-else, multi-way if, and nested if statements. Starting with if-else, essentially it works off of an individualistic linear flow starting with the top if statement and then only running to the else if the first if case is not true and looks like the following:

```
if (userRow < 1)
{
    System.out.println("Invalid row, program ending");
} else if (userRow <= 15)
{
    System.out.println("understood your ticket will cost $" + rowOne);
} else if (userRow <= 30)
{
    System.out.println("understood your ticket will cost $" + rowTwo);
}
```

Essentially what it's doing is cascading if statements that only operate if the previous statement isn't true, following the logic that if this first statement isn't true then go to the else.

A multi-way if statement looks a little different. Essentially it's a bunch of individual statements that will run regardless of if the previous statement worked. Essentially all that's missing is the else from the code line.

And nested if statements are essentially one big if statement with more inside. This works best if when the first original of statement is true you want if conditionals to follow only after the first has been met.

Use of Switch statements for multi-way selection

Switch statements are like if statements but are more case specific. You can't exactly enter a range of values like with an if statement. For example, `(10 >= numOne)` would work for an if statement but is too ambiguous for a switch case. Switch cases work best for checking one true or false value. For example, `if(joyIsTrue) joyIsTrueJoy` is true, being a true/false statement. Meaning that the logic only has to account for one Boolean expression so that the switch can work properly.

Module 3: Loops and Methods

Add the information to demonstrate the following objectives and organize with headings:

- Compare different types of loops (for, while, do-while).
- Use sentinel values and boolean flags to control loop execution.
- Distinguish scope of variables in control structures and methods.
- Use loops and nested control structures to control program flow.
- Write and call methods to pass and return values.
- Trace memory during method calls to evaluate stack frame memory and pass-by-value.

Heading 2 Example

Heading 3 Example

Module 4: Arrays

Add the information to demonstrate the following objectives and organize with headings:

- Declaring, allocating, initializing 1D arrays.
- Reference variables, array memory, and out-of-bounds issues.
- Passing arrays by reference to and from methods
- Trace memory during method calls to distinguish reference variables vs. primitive variables, stack vs. heap memory allocation.
- Design algorithms for processing arrays.
- Designing methods that operate on arrays using loops.

Heading 2 Example

Heading 3 Example

Module 5: Classes and Objects

Add the information to demonstrate the following objectives and organize with headings:

- Use classes to define blueprints for creating objects that combine data (fields) and behaviors (methods).
- Define and implement constructors, including default, parameterized, and overloaded constructors, to initialize objects in different ways

- Encapsulation using access modifiers and getters/setters to protect and manage private data.
- Reference variables for objects, stack/heap diagrams.
- Arrays of objects
- Simple file input/output
- Design Implement and Test Java classes that model real-world problems

Heading 2 Example

Heading 3 Example